MICROSERVICES

Study Notes

Chapters 1 to 3

Toward a Microservices Architecture | Designing a Microservices Operating Model | Designing Microservices: The SEED(S) Process

Source: Microservices: Up and Running
Ronnie Mitra and Irakli Nadareishvili (O'Reilly)

All content in these notes is drawn only from the prescribed textbook.

Contents

1. Toward a Microservices Architecture

- What Are Microservices?
- Reducing Coordination Costs
- The Hard Parts
- Learning by Doing
- Decisions, Decisions...

2. Designing a Microservices Operating Model

- Why Teams and People Matter
- Introducing Team Topologies
- Designing a Microservices Team Topology

3. Designing Microservices: The SEED(S) Process

- Introducing the Seven Essential Evolutions of Design for Services
- Identifying Actors
- Identifying Jobs That Actors Have to Do
- Discovering Interaction Patterns with Sequence Diagrams
- Deriving Actions and Queries from JTBDs
- Describing Each Query and Action as a Specification with an Open Standard
- Getting Feedback on the API Specification
- Implementing Microservices
- Microservices Versus APIs

Quick Revision Sheet

- Key terms
- Key decisions
- Likely exam questions

1. Toward a Microservices Architecture

Goal of the book: to help the reader build a working microservices architecture, using opinionated and prescriptive advice drawn from real practitioner experience (both successes and failures). The authors refine these lessons into a model that gets you "up and running" faster.

Background: the term *microservices* emerged in the early 2010s to describe a style of software architecture in which applications are built from small, independent components that work together. Adoption has grown rapidly. The book cites a market forecast of growth from $2.07 billion to $8.07 billion USD by 2026.

Key tension: microservices are hard. Making many independent parts work together is harder than it sounds, and management, maintenance, support and testing costs add up at scale. But done well, microservices let you make software changes **faster and safer at scale**, which gives the business agility.

Approach: most early adopters reached microservices through iterative experimentation and continuous improvement rather than a plan. The book lets you start from the patterns, methods and tools that have already been proven together, then tune them to your organization.

1.1 What Are Microservices?

There is no single official definition. The book looks at two.

Definition 1: Lewis and Fowler (2014)

> **Lewis and Fowler, in essence**
>
> Microservices are an approach to developing a single application as a **suite of small services**, each running in its own process and communicating with lightweight mechanisms, **built around business capabilities** and **independently deployable** by fully automated deployment machinery.

The heart of their article is **nine characteristics** of microservices. The textbook names these:

- Componentization via services (the core characteristic: breaking an application into smaller services)
- Organization around business capabilities
- Decentralized governance
- Infrastructure automation
- Products not projects
- Smart endpoints and dumb pipes
- Design for failure
- Evolutionary design

Together these cover technology, infrastructure, engineering, operationalization, governance, team structure and culture, a very large set of concerns. (The book groups these as organization/management design, DevOps and Agile delivery practices, and architecture principles.)

Definition 2: Nadareishvili, Mitra, McLarty and Amundsen (Microservice Architecture)

> **Definition**
>
> A **microservice** is an independently deployable component of bounded scope that supports interoperability through message-based communication. **Microservice architecture** is a style of engineering highly automated, evolvable software systems made up of capability-aligned microservices.

This is similar to Lewis and Fowler but emphasizes **bounded scope**, **interoperability** and **message-based communication**. It also separates the microservices (the components) from the architecture that enables them.

Three general design traits

Because definitions vary and it is hard to tell whether you have built a "textbook" microservices system, the authors use the label to describe three general design traits:

1. The application architecture is primarily composed of machine-invocable "services" made available on a network.
2. The size (boundaries) of services is an important design factor. Boundaries include runtime, design-time and people factors.
3. The software system, the organization and the way of working are holistically optimized to achieve a goal.

These traits do not prescribe organizational styles, tools or formal patterns. They give just enough to recognize a microservices system. The authors argue that **why** you build microservices matters more than **what** they are, and the best reason is to **reduce coordination costs**.

1.2 Reducing Coordination Costs

Practitioners almost universally report faster software delivery. The textbook attributes this to the fundamental benefit of the style: a **reduction in coordination costs**.

Speed can be gained other ways, such as cutting corners and incurring technical debt, or neglecting stability and safety. Those can be reasonable for some businesses. But the financial, healthcare and government sectors cannot compromise on safety for speed, while still facing market pressure to move quickly. Microservices offer a way to **increase speed without compromising safety, at scale**.

The coordination cost problem

Complex software needs many people and a lot of time. Multiple teams build different parts of a system, on independent roadmaps and at independent paces. Periodically the parts must be integrated to resolve dependencies, and at that point the otherwise autonomous teams must coordinate (Figure 1-1 in the book shows a timeline with coordination touchpoints).

> **The Jane and Tyrone example**
>
> Jane leads the Accounting workstream. Her team finishes a sprint and depends on a component being built by the Shipment team, led by Tyrone. Because roadmaps are independent, Tyrone's component may not be ready. Jane has two choices:
>
> **1. Wait** for the component and do a proper integration test. This prioritizes **safety** but sacrifices **speed**, because her team is halted.
>
> **2. Rely on an agreed interface contract** and proceed assuming the component will arrive as planned. This keeps **speed** but risks **safety**, since integration testing did not happen early and a "happy path" assumption was made.

In a sufficiently complex system these choices occur often, producing a pronounced **tension between speed and safety**. Since coordination costs cause the tension, the fix is to design a system that **minimizes coordination costs**, so teams do not face the choice most of the time. This needs **autonomous teams working on small batches of isolated work**, which is what microservices architecture is about in essence.

> **KEY DECISION: A universal litmus test**
> When in doubt about a design decision, ask: "Is this decision I am facing going to reduce coordination costs for my teams or not?" Viewing decisions through coordination cost makes the right answer more obvious.

Microservices let companies in complex domains have the **agility of a smaller, simpler company** while still using the power and reach of their real size. The benefits do not come free; they need up-front work, focus and decision making.

1.3 The Hard Parts

First-time adopters struggle with the enormous scope and breadth of a microservices system: beyond small bounded services you must work out infrastructure, data models, frameworks, team models and processes. The book names three big design problems:

Problem	Explanation
Long feedback loops	Impactful decisions are not easy to measure. Problems may emerge only much later. *Example:* choosing a shared communication library to ease service-to-service communication; later, keeping that library up to date across all services and teams becomes a huge problem. It is hard to see the impact of a decision until problems arise, so options are hard to evaluate.
Too many moving parts	A microservices system is a complex adaptive system: each part affects the others, and together they produce emergent behavior. *Example:* introducing Docker containerization inevitably changes the development and release life cycle. Consequences can be planned or unintended, so it is hard to predict the impact of changes and there is a risk of doing more harm than good.
Analysis paralysis	Long feedback loops plus a complex system mean decisions are both highly impactful and hard to measure. This leads to endless speculation and discussion out of fear of building the wrong system, instead of delivering business outcomes. The web full of horror stories, "bumper sticker" advice and contradictory best practices makes it worse.

The real challenge is dealing with a big, complicated system across a huge scope. The book responds by bringing together practices and patterns evolved for this type of domain, and tools that make the work easier, safer, cheaper and faster.

1.4 Learning by Doing

Many successful implementers built their systems through continual iteration and often through failed architectures first. With unlimited time one could learn by experimentation alone, but most people lack that luxury. The book therefore provides a **prescriptive microservices model** with decisions on team design, process, architecture, infrastructure, and even tools and technologies. Following it, the reader ends up with a simple, operational microservices system on a cloud-based architecture.

> **Running example: a fictional airline reservations system**
> A vastly simplified system with two functions: a **read-only flight information service** and a **seat reservation service**.

The authors do not expect readers to agree with every decision; questioning them is part of the learning. The approach relies on the **Dreyfus five-stage model of adult skill acquisition**, whose first stage is following prescriptive guidance before proficiency and expertise develop.

The "Up and Running" microservices model: scope of the book

Area	Chapters	What is covered
Team design	2	People side of microservices: team design challenges, factors affecting coordination, example teams, and the Team Topologies tool.
Microservice design	3, 4	The SEED(S) design process (Ch. 3); service boundaries using Domain-Driven Design concepts and Event Storming to "rightsize" services (Ch. 4).
Data design	5	Data factors in a microservices system; data independence.
Cloud platform	6, 7	Immutable infrastructure and infrastructure as code, AWS, GitHub Actions CI/CD (Ch. 6); networking, Kubernetes cluster and GitOps deployment tool (Ch. 7).
Microservices development	8, 9	Principles and tools (Ch. 8); two independent, heterogeneous microservices (Ch. 9).
Release and change	10, 11	Deploying a service using DockerHub, Kubernetes, Helm and Argo CD (Ch. 10); retrospective on the system (Ch. 11).

The model rests on five guiding principles, including the **twelve-factor app** pattern (documented in the book's GitHub repository).

1.5 Decisions, Decisions...

The quality of software and of business outcomes depends on the quality of decisions. Decisions may become outdated when technology, people or situations change, and some were never good. We therefore need a way to **capture important decisions so they can be re-evaluated and improved**. The tool used is the **architecture decision record (ADR)**.

Four elements of a good decision record

Element	What it captures
Context	The challenge or problem, and the constraints. Lets us understand the rationale and why the decision may need updating.
Alternatives	The choices available. A decision is not a decision unless there is a choice; this shows the "selection space" at the time.
Choice	The option actually selected.
Impact	Consequences and trade-offs: how the choice affects ways of working or other decisions.

Records can be text files, a project management tool or a spreadsheet; the content matters more than the format or tooling.

Lightweight Architectural Decision Record (LADR)

The book uses **LADR**, created by **Michael Nygard**. It is concise, written in **Markdown** text files, and can be managed in version control like source code. Markdown is readable raw and is rendered by tools such as Confluence, GitLab, GitHub and SharePoint.

Structure and sections:

- **Title** with a short code. Example: "OPM1" means the first decision related to the operating model.
- **Status**: the life-cycle stage, e.g., Proposed, Accepted, Under Review.
- **Context**: problem, constraints and background.
- **Decision**: the choice made, with alternatives considered.
- **Consequences**: trade-offs and impacts.

Example from the book (the first decision is to keep decision records):

```
# OPM1: Use ADRS for decision tracking
## Status
Accepted
## Context
A microservices architecture is complex and we'll need to make many
decisions. We'll need a way to keep track of the important decisions we
make, so that we can revisit and re-evaluate them in the future. We'd
prefer a lightweight, text-based solution so that we don't have to
install any new software tools.
## Decision
We've decided to use Michael Nygard's lightweight architectural decision
record (LADR) format. We'll keep each LADR record in its own text file
and manage the files like code.
Alternatives considered:
* Project management tooling (not selected, no new tools wanted)
* Informal "word of mouth" record keeping (not reliable)
## Consequences
* We'll need to write decision records for key decisions
* We'll need a source code management solution to manage the files
```

> **KEY DECISION: Use ADRs for decision tracking**
>
> Use ADRs to log the key decisions made in system design and build. Throughout the book, key decisions are flagged in notes like this one.

1.6 Chapter 1 Summary

- A loose definition of a microservices system with three key traits.
- **Reducing coordination costs** is identified as the key benefit.
- Complexity and analysis paralysis are the main challenges for adopters.
- The prescriptive "up and running" model accelerates learning.
- ADRs (specifically LADR) are used to record decisions.

2. Designing a Microservices Operating Model

Success with microservices takes more than writing and deploying code: you need the right people, ways of working and principles. The chapter designs an **operating model** for the example application.

> **Definition: operating model**
> The set of **people, processes and tools** that underlies your system. It shapes all decision making and work, and can define team responsibilities and governance over decisions and work. Think of it as the "operating system" for your solution.

The book limits scope to the most important parts: **how teams are designed and how they work together**. It introduces **Team Topologies** and produces a team-based design that is the foundation for the rest of the build. (You do not need to actually assemble these teams to follow the book's build.)

2.1 Why Teams and People Matter

The goal is to put good technology in the hands of **independent, high-functioning teams**. Technology alone is useless without the right culture and organization. A microservices implementation is valuable because it makes change easy and quick, but change is a by-product of an organization's **decision-making capability**. If you cannot make quality decisions quickly, you will struggle to get value; the book compares it to a racing car with a very poor engine.

Conway's Law

> **Conway's Law (attributed to Fred Brooks in the book)**
> Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.

Mel Conway made the original observation in "How Do Committees Invent?". **Example:** a team that must consult a centralized team of database experts for every data-model change will likely end up with a centralized data model and implementation. The system ends up matching the organization and its coordination model.

Three people factors have the biggest impact on a microservices system: **team size, team skills and interteam coordination**.

Team size

Too many people means more internal communication and coordination, which slows delivery. Too few means not enough minds and hands. "Rightsizing" teams is part of system design. Three supporting ideas in the book:

Idea	Point
Bill Gore (W. L. Gore)	Everyone on a team must have a personal relationship with each other. When members no longer know each other, the unit is too large.
Dunbar number	Humans can comfortably maintain about 150 stable relationships, about 5 intimate relationships and about 15 trusted friends.
Two-pizza rule (Jeff Bezos)	An Amazon team should be small enough to be fed with two pizzas, probably around 5 to 15 people.

All of these point to a size limit based on the ability to communicate effectively. The book sets team size at **five to eight people**.

> **KEY DECISION: Team size should be limited**
> The teams that perform work in the system should have no more than eight people each.

Knock-on effects: smaller teams usually mean more teams, so the rest of the design must enable independent, autonomous work (small teams that spend all their time coordinating are no use). It also limits the number of specialists per team, so collective talent must still be sufficient.

Team skills

A team is only as good as its members. Questions: which roles are needed, how experienced should members be, what mix is right? The answers depend on the organization (e.g., a company hiring the top 1% of global talent versus one hiring locally and growing people on the job will design teams differently). The book adopts the principle of the **cross-functional team**: people with different expertise (technology and business, e.g., UX designers, application developers, product owners, business analysts) working toward the same goal. A big advantage is **better decisions made faster**.

Rather than prescribe exact roles, the book makes two general decisions: (1) teams should be **cross-functional**, since microservices work better when teams can make good decisions on their own; (2) teams should have members who **directly influence the output**, with no observers or tangentially related people.

> **KEY DECISION: Principles for team membership**
> Teams should be cross-functional and consist only of members who can add value to the team's deliverable, service or product.

Interteam coordination

Communication **among** teams, rather than inside them, is what can really bog down a microservices system. Reducing it lets teams deliver faster. Yet complete independence is not practical:

- Coordination is needed to establish shared goals, communicate change requests, deliver feedback and resolve problems.
- Fully independent teams lose the chance to share. Efficiency becomes local to each team, e.g., if every team designs its own cloud network architecture, the work is not done once and shared.
- Too much independence causes system-level inefficiency and misalignment with organizational goals; too much coordination loses the benefits of changeable microservices.

The challenge is to **balance independent work and coordinated effort**, with experimentation and continuous tuning. The book mentions Fred George's **Programmer Anarchy** (teams fully self-organize, choose work and design solutions) but notes most enterprises would struggle to do this consistently.

Optimizing coordination needs **active design effort**. A common mistake is to focus only on technical architecture, so team design forms around the technology and coordination problems appear too late to fix cheaply. The remedy is to design teams and coordination **first**, sometimes called the **inverse Conway maneuver**: the communication structure you design will shape the system that gets created.

> **KEY DECISION: When to design teams and coordination models**
> Team and coordination design should start before the design of the system architecture or microservices. The team and coordination models must continually be updated and improved for the life of the system.

2.2 Introducing Team Topologies

Team Topologies is a design approach by **Matthew Skelton and Manuel Pais**. It provides a formal language for team design with special focus on how teams work with each other. The book uses three elements: **team types, team interaction modes and diagramming**.

Four team types

Archetypes that describe a team's basic nature from the perspective of its communication with the rest of the organization.

Team type	Description	Example in the book
Stream-aligned	Owns and runs a deliverable piece of work and continually delivers something relevant to the business. Embodies "You build it, you run it" (Werner Vogels). Does not disband after a release; owns a "stream" of changes, improvements and fixes.	Microservices teams continually releasing features to services they own.
Enabling	Supports other teams with a consulting engagement model. Specialists and subject-matter experts bridge gaps in expertise or help others see the bigger picture.	An enabling architecture team helping teams understand emerging technical standards and conventions.
Complicated-subsystem	Works on a domain that is difficult to understand, where expertise is scarce and cannot be embedded in every team.	A security team for cryptographic tuning, engaging with individual teams as needed.
Platform	Supports other teams like an enabling team but delivers a self-service experience. Invests in tools and processes that scale; the platform becomes a product whose users are the other teams. Needs up-front investment and continual support.	Operations team offering build and release tools to development teams.

Key contrast: enabling and complicated-subsystem teams are limited by the bandwidth of their people; a platform team scales through self-service tooling.

Three interaction modes

These describe different levels of coordination between teams.

Mode	Description	Example in the book
Collaboration	Both teams work closely together. Good for learning, discovery and innovation, but needs high coordination and is difficult to scale.	A security team and a microservices team designing, writing and testing code together for a more secure version of the software.
Facilitating	Like collaboration but unidirectional: one team plays a support role to help the other deliver its outcome.	An infrastructure team helping a microservices team troubleshoot issues with the network architecture provided.

Mode	Description	Example in the book
X-as-a-service	Consumer-provider relationship with minimal coordination. A team releases a shared process, document, library, API or platform. Scales well; natural fit for platform teams.	An enabling architecture team documenting recommended software patterns for all teams ("patterns as a service").

2.3 Designing a Microservices Team Topology

The Team Topologies language is built for **visual representations**. The outcome is a diagram of the teams needed and their key interactions. The book follows five steps:

1. Establish a system design team.
2. Create a microservices team template for future teams.
3. Define platform teams.
4. Add enabling and complicated-subsystem teams.
5. Add key consumer teams.

For each step: identify one or more teams, create and populate a **team design document** (type, size, responsibilities) and draw the key interactions. The book stresses there is no single topology that fits everyone; this one is a consolidated version of large enterprise-scale implementations that have worked well.

Step 1: System design team

A designated group that shapes the vision and behavior of the system. Three core responsibilities:

- **Design team structures**: it designs the teams that will build the system (the work done in these steps).
- **Establish standards, incentives and "guardrails"**: shape what teams can decide so results align with system goals. Too much standardization is hard to maintain and too restrictive, so good designers use **incentives** to encourage desired behavior and **guardrails** as lighter recommendations and references rather than hard rules.
- **Continually improve the system**: improve team designs, standards, incentives and guardrails, and monitor or measure the system as a whole.

Team type: Enabling (its work is mostly consulting and facilitating delivery teams after initial setup). **Size:** 3 to 5 people (smaller than the general limit): a few senior leaders, architects and system designers who can decide quickly. Team design documents are kept as Markdown files (e.g., system-design-team.md) so they can be versioned like code.

Step 2: Microservices team template

In this model **every microservice is owned by a team**, which owns the decisions and work of designing, building, delivering and maintaining it. One team may own several microservices, but responsibility for a microservice is **never shared across multiple teams**.

> **KEY DECISION: Microservice ownership**
>
> Each microservice will be owned by a single team, who will design, build and run it. This team is responsible for the microservice for the lifetime of the service.

Rather than designing each team individually, a **template** is defined (a "cookie cutter"; in programming terms a "class" from which "instances" are created). Type: **Stream-aligned**. Size: **5 to 8**. Responsibilities: designing and developing microservice(s); testing, building and delivering them; troubleshooting issues.

Diagram: the microservices team is a horizontal yellow rectangle. The system design team is a vertical violet rectangle placed to its left, showing a facilitating interaction. Each team type gets its own color. The generic box is later replaced by real team and service names.

Step 3: Platform teams

Stream-aligned teams would otherwise have to solve many development, testing and implementation problems themselves. Platform teams make common components available **as a service**, which improves scalability and reduces the coordination problems usually seen with centralized shared components.

The **cloud platform team** offers network, application and deployment infrastructure so other teams can create new environments on demand. **Type:** Platform. **Size:** 5 to 8. **Responsibilities:** design and develop network infrastructure; design and develop application infrastructure; provide tools for building a new environment; update network and application infrastructure when required.

The last responsibility is key: the platform team must treat its users as **customers** and keep improving the offering. In the diagram it is a light blue horizontal box below the microservices teams and connected to the system design team. A small **black arrow** shows the **x-as-a-service** interaction with the microservices team, and the system design team enables the platform team so the platform fits the system vision. In practice more than one platform team may be needed, and their coordination must also be considered.

Step 4: Enabling and complicated-subsystem teams (the release team)

Additional teams may own capabilities needing special skills or complicated features. The model adds a **release team** that releases (deploys) microservices into a production-like environment. Most organizations need an extra testing and acceptance check before go-live, so microservices teams deliver a **built and tested container**, which the release team automatically deploys while coordinating tests, approvals and deployment.

Type: Complicated-subsystem (specialist knowledge of the release, approval and deployment process, collaborating with stream-aligned teams). **Size:** 5 to 8. **Responsibilities:** releasing microservices to production; coordinating approvals for releases. In the diagram it is a red square near the end of the microservices team's box.

> **Trade-off**
>
> The release team brings **coordination costs**. At scale (e.g., daily releases across many microservices) it will struggle to coordinate everything. If that happens, change the team design and **shift deployment responsibility to the individual microservices teams**.

Step 5: Consumer teams (the API team)

Microservices are only useful if used. In this model the main consumer is the **API team**, which exposes the microservices to other development teams as an API (for example, a mobile team uses the API and never calls microservices directly). **Type:** Stream-aligned, because the API must continually change with business needs and consumer demand. **Size:** 5 to 8. **Responsibilities:** design, develop and maintain APIs at the boundary of the system; connect the API to internal microservices.

Because the API calls microservices, the API team **depends on** the microservices team. This is modeled with a rectangle at the top of the diagram (same yellow, as it is also stream-aligned) and a black arrow for **x-as-a-service**: the microservices team must make services invocable and usable in a self-service fashion.

Summary of the example team topology

Team	Type	Size	Responsibilities
System design team	Enabling	3 to 5	Design team structures; establish standards and guardrails; continually improve the system.
Microservices team (template)	Stream-aligned	5 to 8	Design and develop microservice(s); test, build and deliver; troubleshoot issues.
Cloud platform team	Platform	5 to 8	Network and application infrastructure; tools for building a new environment; update infrastructure.
Release team	Complicated-subsystem	5 to 8	Release microservices to production; coordinate approvals.
API team	Stream-aligned	5 to 8	Design, develop and maintain APIs at the system boundary; connect the API to microservices.

2.4 Chapter 2 Summary

- The decisions, team definitions and topologies together form the **microservices operating model**. Every later decision is heavily influenced by it.
- The design is lightweight. In practice, draw more than one topology to show different interaction modes (e.g., troubleshooting needs a different engagement model), and diagram the interactions needed to change what the cloud platform, system design and release teams provide.
- Team definitions and topologies are documents that can be **versioned like code**. Extra assets could include SLAs for platform teams and skill inventories for stream-aligned teams.
- The operating model should be **continually improved**.
- The model enables fairly independent, autonomous work, but needs investment in building a cloud platform as a service.

3. Designing Microservices: The SEED(S) Process

The main benefit of microservices is increased development speed without compromising safety, at scale. This happens through **conscious design**, not by accident. In all but the simplest cases it is impossible to iterate toward a successful microservices architecture without an effective, explicit, end-to-end system design.

This chapter introduces an **evolutionary** design process first formulated by one of the authors at a healthcare startup and later used on many projects, from small organizations to ones with thousands of engineers across hundreds of teams.

> **KEY DECISION: Use a standard process for service design**
> Use a standard, repeatable process to achieve consistently high-quality, customer-centric design for the services in your system.

3.1 Introducing the Seven Essential Evolutions of Design for Services (SEED(S))

SEED(S) is a **top-down, multistep methodology**, a collection of reusable processes in which each later step **evolves** from the previous one; hence the name *Seven Essential Evolutions of Design for Services*. The tongue-in-cheek name reflects that the analyses are the "seeds" from which a complex microservices system grows.

Why it matters: microservices are independently deployable components accessible over standard network protocols (e.g., a web service request or RPC). Exposing components as services means committing to **explicit public interfaces**, and good interface design strongly affects architectural robustness and developer productivity. SEED(S) is a repeatable, battle-tested way to design interfaces that are user-friendly and robust.

It is also useful beyond microservices, e.g., for RESTful and GraphQL APIs for frontend UIs. This is unsurprising because **a microservice is also a kind of API**, developed with a specific type of boundary in mind: one that minimizes coordination needs.

The seven steps

#	Step	Purpose
1	Identifying actors	Define who the service is for (customer-centric scoping).
2	Identifying jobs that actors have to do	Capture what each actor needs to get done (jobs to be done, in Job Story format).
3	Discovering interaction patterns with sequence diagrams	Model how requirements translate into technical interactions.
4	Deriving high-level actions and queries from JTBDs and interaction patterns	Turn jobs into developer-friendly interface contracts.
5	Describing each query and action as a specification with an open standard (e.g., OpenAPI, GraphQL schemas)	Formalize the contract before coding.
6	Getting feedback on the API specification	Validate usability with client developers.
7	Implementing microservices	Code only after the design is validated.

3.2 Identifying Actors

SEED(S) takes a distinctly **customer-centric** approach and treats designed services as **products** ("APIs are products"). Product management as a field predates APIs and the internet (the book traces it to Procter & Gamble and Neil H. McElroy in the 1930s), so its techniques can be reused for API and service design.

To design a product you must understand the "customer". In API and service work the term **actor** is used instead, avoiding any accidental connotation of a financial transaction between consumer and publisher. Actors are inspired by **user personas** from interaction design (introduced by Alan Cooper in *The Inmates Are Running the Asylum*, 1998), but are not identical to them: their purpose is to aid modeling when actual user data is typically limited.

Main motivation: scoping and prioritization. Typical problems in API design are **over-abstraction** and **lack of clarity about user needs**, e.g., APIs that merely expose database tables over HTTP or give networked access to application internals via RPCs. These are designed from the publisher's goals, not the consumer's. SEED(S) fixes this "upside-down" problem by asking first: who will use this API and what do they need?

> **KEY DECISION: Scope service design using key actors**
> Start service design by identifying key actors in your domain, to achieve customer-centric scoping of the capabilities represented by the services.

Four rules for identifying the right actors

1. **Specific more than precise.** Identify the key traits that differentiate actors rather than excruciating detail. Modeling is by definition imprecise; capture the prioritized view of reality relevant to you.
2. **Avoid overlapping or too-broad definitions**, which are red flags. Actors must be defined **in context**. A company-wide "portfolio" of actors reused for every application is a sure sign the process has derailed.
3. **Represent needs, pain points and behaviors.** What distinguishes one actor type from another should be relevant, with very limited overlap.
4. **Less is more.** Use as few actors as possible, but no fewer than necessary. More than five actors for a service may indicate prioritization gone awry or service boundaries that are too broad.

Example actors in the airline project

Actor	Description
Frequent flyer (Emma)	Travels for work, has elite loyalty status, manages travel through her work's reservation system, uses several connected apps. Eligible for many perks; plans trips at short notice; with family, typically uses loyalty miles.
Family vacationer (Riley)	Travels mostly for vacations with spouse and kid(s); plans well in advance; travels infrequently.
Airline customer service agent (Sean)	Experienced agent helping travelers book, rebook and resolve issues during and after travel via phone and online chat.

3.3 Identifying Jobs That Actors Have to Do

After identifying actors, spend significant time understanding the **jobs they have to get done**, and only then create a solution. This rests on the premise that APIs and microservices are products. The book uses Kotler et al.'s broad definition of a product: anything offered to a market for attention, acquisition, use or consumption that might satisfy a want or need, including services. Web APIs and microservices fit: producers offer services, consumers' needs are satisfied, and supply and demand create a "market".

Consumers: of APIs are typically frontend (web, mobile) or third-party partner applications; of microservices are other parts of the system itself. This distinction is largely irrelevant to the design process.

Problem-focus, not solution-focus: many products are designed by providers obsessing over what they offer rather than the problems customers need to solve. Theodore Levitt's famous point: people do not want a quarter-inch drill, they want a quarter-inch hole. The real job may be hanging a picture, so an inventor with a simpler alternative may outmaneuver a drill-perfecting company. **Problems are timeless; solutions change** (nobody uses floppy disks, but the job of saving and transporting data remains).

Clayton Christensen's **theory of jobs to be done (JTBD)**: customers often buy things because they find themselves with a problem they want to solve, and designs succeed when **the job, not the customer, is the fundamental unit of analysis**.

> **KEY DECISION: Use jobs as the unit of analysis**
> Use jobs that key actors have to get done, in your domain, as the unit of analysis for collecting requirements.

Job Story format

JTBDs are captured in a standard format, the **Job Story** defined by **Paul Adams**:

> **Job Story template**
> **When** <a circumstance>, **I want to** <motivation>, **so I can** <goal>

A Job Story centers on **circumstances**, the actor's **motivation** and the **goal**. Adams' original is first person; SEED(S) prefers **third person** to highlight who the actor is.

Job Story versus User Story

They look similar but differ crucially (Alan Klement). **User Stories** revolve around a persona ("As a <persona>..."); **Job Stories** disregard the persona and emphasize the **circumstance**. Persona does not matter for a specific job: to hang a painting, a licensed contractor and a novice both need a quarter-inch hole. So actors are used to **scope the list of jobs**, but when describing each job, identify circumstances rather than repeating "as a frequent flyer...".

> **KEY DECISION: Use the standard Job Story format**
> Use a standard format for capturing JTBDs to uniformly capture circumstances, motivations and goals for all jobs.

Example JTBDs in the airline project

Actor	Job Stories
Family vacationer (Riley)	1. When Riley is planning a flight for their family vacation, they want to filter available flights by multiple criteria (four adjacent seats, number of connections, connections through child-friendly airports, etc.), so that their family can fly with maximum comfort. 2. When Riley is planning a quick, unplanned long-weekend getaway, they want suggestions for interesting, affordable, short-flight trips, so they have a list of choices to consider.
Frequent flyer (Emma)	1. When Emma's plans change and she cannot travel on a booked flight, she wants to easily reschedule, so she can get a flight that works for her new plans. 2. When Emma prefers an available seat other than the one assigned, she wants to select the alternative seat, so she can enjoy her flight more.
Customer service agent (Sean)	1. When a customer calls Sean, he wants a servicing ticket open and pre-filled with customer information, so he can track progress toward resolution. 2. When a customer asks Sean to find a convenient flight, he wants to find a fitting flight using flexible filtering criteria, so he can meet the need and book a flight.

Job Stories should be derived from **user research** where possible. They suit conversations with subject matter experts and customers, but they are not convenient for deriving technical requirements, so they must be translated into a developer-friendly format in the next steps.

3.4 Discovering Interaction Patterns with Sequence Diagrams

Job Stories are written by product managers from a business-value perspective and rarely map directly to services. To design well, we must understand the **service interaction patterns** of the subdomain. For complex interactions a linear list of Job Stories is insufficient, so an **interaction diagram** shows the sequence of events.

SEED(S) recommends **UML sequence diagrams** (any approach is acceptable since the purpose is to communicate the model), preferably in a text-based format such as **PlantUML**, because modeling is a team activity. Text-based diagrams let team members:

- Keep modeling **independent of anyone's editor** (PlantUML is supported in tools like Confluence; free editors include LiveUML and PlantText).
- **Version-control** diagram sources: text is easy to diff, merge and review in pull requests, unlike binary graphics.
- **Integrate modeling into the release process**: diagrams become code and can be handled like code, e.g., in Git.

> **KEY DECISION: Use PlantUML sequence diagrams to discover interaction patterns**
> Use UML sequence diagrams expressed in a textual (Markdown) format such as PlantUML.

At this point the aim is a **technical model** for requirements gathered in the physical world. Jobs and actors do not map one-to-one to technical interactions, so events in the diagram need not occur between the actors from step 1 or correspond directly to jobs; the diagram may go a level deeper and show interactions between services. Example from the book:

```
@startuml
actor Agent
```

```
participant "Agent Servicing" as AS
participant "Reservations API" as rAPI
participant "reservationCRUD" as rCRUD
AS -> rAPI: checkRes(reservationId)
rAPI -> rCRUD: reserve(data)
rAPI -> rCRUD: cancel(reservationId)
@enduml
```

Here **Agent Servicing** is a web or mobile UI used by agents, **Reservations API** is a REST or GraphQL API the app calls, and **reservationCRUD** is one of the microservices behind the API.

3.5 Deriving Actions and Queries from JTBDs

With interaction patterns understood, jobs are transformed into technically oriented **interface contracts**. Following Bertrand Meyer's **command query separation (CQS)** principle, SEED(S) models interfaces as two distinct kinds of interaction: **actions** (the "commands" in CQS) and **queries**.

> **KEY DECISION: Separate service endpoints into commands and queries**
> Use the CQS principle to model the action side of services separately from the query side, and document each with its own standard format.

	Query	Action
Nature	A lookup with defined inputs and outputs; a clear contract between client and server.	A request that causes a state modification; its whole purpose is side effects.
Side effects	None. Does not modify system state.	Yes, by design.
Template	Description of the query Input: list of input variables Response: list of output data elements	Description of the action Input: list of input variables Expected outcome: the induced side effect Response (optional): data elements, if any

> **Important: mapping is not 1:1**
> A Job Story does not always yield exactly one query or action. It can translate into **multiple** queries and actions, and one query or action may combine **multiple** Job Stories. SEED(S) is a process of modeling, design and discovery, not a robotic process; human judgment remains.

Example queries from the airline project

Query 1: Flight Search (from Riley's job of finding comfortable flights)

- Input: departure_date, return_date, origin_airport, destination_airport, number_of_passengers, baby_friendly_connections, adjacent_seats, max_connections, minimum_connection_time, max_connection_time, order_criteria [object], customer_id (optional, to check loyalty privileges)
- Response: list of flights satisfying the criteria

Query 2: Lookup of Alternative Flights for a Date Change (from Emma's rescheduling job). The reservation id lets the system retrieve origin, destination and preferences so the traveler need not re-enter them.

- Input: reservation_id, new_departure_date, new_return_date
- Response: list of alternative flights

Example actions from the airline project

Travel Rebooking

- Input: original_reservation_id, new_flight_id, seat_ids[]
- Expected outcome: new flight booked or error returned; if the new flight is booked, the old one is canceled
- Response: success code or a detailed error object

Seat Change

- Input: reservation_id, customer_id, requested_seat_ids[]
- Expected outcome: new seat reserved if available and the traveler is qualified; old seat canceled if the new seat is successfully reserved
- Response: success code or a detailed error object

> **Alternative: Microservice Design Canvas**
> For sophisticated cases where actions and queries are not enough, the book recommends **Matt McLarty's Microservice Design Canvas**. The canvas and the actions-and-queries analysis are **substitutable techniques** in the same SEED(S) phase. The canvas is more powerful but not covered in the book.

3.6 Describing Each Query and Action as a Specification with an Open Standard

Before coding, formally describe the **interface contract** of an API or microservice. A codified contract is a mutually agreed understanding between producer and consumers (API client developers), is easily converted into user-friendly documentation and interactive playgrounds, and, when built on open standards such as the **OpenAPI Specification (OAS)** and **GraphQL**, gets wide tooling support (documentation rendering, developer portals, etc.).

- **OAS** describes RESTful APIs in a standard, tech-stack-agnostic manner. It is governed by the **OpenAPI Initiative**, a Linux Foundation Collaborative Project. The version at the time of writing was **3.0.2**.
- Microservices need not be RESTful. Other options: **GraphQL, gRPC and asynchronous event communication** (e.g., JSON-, ProtoBuf- or Avro-encoded messages on Kafka Streams). Whichever style is used, **messages and their formats should be documented as part of the contract**, and SEED(S) can be applied to each style. REST is used in the book because it is the easiest and most ubiquitous.
- **Tooling suggestion:** VS Code with the OpenAPI Designer plug-in; with a YAML descriptor open, use the OAS preview command to see the rendered specification.

Example OAS for the rebooking action

The book designs a RESTful endpoint for the rebooking action. Its main parts:

OAS element	In the example
openapi / info	OpenAPI 3.0.0; title "Airline Reservations Management API"; version 1.0.1.
servers	http://api.example.com/v1 (Production Server).
path and method	PUT /reservations/{reservation_id}: "Book or re-book a reservation".
parameters	reservation_id in the path, required, string: the unique identifier of the reservation created or changed.

OAS element	In the example
requestBody	JSON object with outbound and returning objects, each containing flight_num, flight_date and seats[].
responses	200: successful reservation, returns a reservation_id. 403: seat(s) unavailable, booking failed, with detailed information.

```
openapi: 3.0.0
info:
  title: Airline Reservations Management API
  version: 1.0.1
servers:
  - url: http://api.example.com/v1
    description: Production Server
paths:
  /reservations/{reservation_id}:
    put:
      summary: Book or re-book a reservation
      parameters:
        - name: reservation_id
          in: path
          required: true
          schema: { type: string }
      requestBody: # outbound / returning: flight_num, flight_date, seats[]
      responses:
        '200': # Successful Reservation (returns reservation_id)
        '403': # seat(s) unavailable. Booking failed.
(abridged from the book's fuller listing)
```

Producing a formal API contract is a huge milestone, but good API design does not end here.

3.7 Getting Feedback on the API Specification

The draft design must be shown to the **client developers** who will use the APIs and services, and their feedback collected. If earlier steps were active brainstorming, this is a stage of **careful listening and reflection**, and it is essential for APIs that stand the test of time and that clients love to use.

> **KEY DECISION: Collect feedback on your service designs**
> Service design is not done until it is presented to the target audience for the service and feedback is collected and applied to the initial designs.

Two groups of customers must be kept in mind:

- **End users of the system**, whose experiences your APIs enable. Interviewed at the start of SEED(S) to collect Job Stories.
- **Client developers** who code against your services (APIs or microservices) and build the end-user experiences (web, mobile apps). Feedback can come as early as the interaction design phase, and again once the OAS is produced and **before coding**, to avoid building something rejected for poor usability.

> **Two studies, two goals**
> Studying end users ensures we **build the right thing**. Studying client developers ensures we **build it the right way**.

3.8 Implementing Microservices

Implementation is **intentionally the last step**. Coding is one of the most expensive engineering activities, and recoding functionality designed on wrong assumptions is horrible, time-consuming and expensive. A carefully thought-out process like SEED(S) before coding saves time and delivers better outcomes. (Implementation of the example services is covered in Chapter 9.)

3.9 Microservices Versus APIs

Microservices and APIs are both capabilities exposed over standard network protocols (most commonly HTTP). Capabilities exposed as HTTP endpoints were known as web APIs long before "microservices" was coined. This raises questions that confuse teams: are microservices just smaller APIs? Do conventional APIs still matter, or do microservices replace them? Developers often call any small, focused API a "microservice", giving it the same role APIs had before. The authors consider this **not ideal** for successful microservices thinking.

> **Microservices are not just smaller APIs**
>
> Microservices provide the **implementation** of your system, while **APIs should still be the outward-facing interface** of a system. If microservices replace anything, it is the **modular components** (statically or dynamically linked submodules) used to build systems; in a microservices architecture the building blocks are networked services.

A similar separation of "internal APIs you build with" and "external APIs optimized for frontends" is described as the **Backend for Frontend** pattern by **Phil Calçado** (SoundCloud), and by **Daniel Jacobson** at Netflix as **Experience (frontend)** and **Ephemeral (backend)** APIs.

> **KEY DECISION: Web APIs are layered on top of microservices**
>
> Differentiate between web APIs that represent the public interface of your subsystem and microservices that represent the implementation of the same system. Avoid thinking of microservices as "just small APIs."

Recommended separation of duties (the authors' opinion)

- All **business logic (capabilities)** is implemented by **microservices**.
- **APIs** act as a **thin orchestration layer** in front of microservices.
- There is **no 1:1 relationship** between an API and the microservices implementing its capability; they belong to fundamentally different architectural layers.
- Avoid microservices directly invoking each other; for loose coupling, implement orchestrating workflows in the API layer so microservices know nothing about each other.

The Unix philosophy analogy

Unix tools are composable (via input/output piping) because they behave the same for any input and do not care who calls them or where output goes. **Components cannot explicitly know about each other if they are to be composable.** Loose coupling, not just small size and focus, makes the whole work. The same holds for microservices.

> **Keep microservices unaware of each other**
>
> Avoid microservices directly "knowing" about each other and calling each other through **synchronous interfaces**. Orchestrate multi-microservice processes in the **API layer**. If that is not possible, use

> **asynchronous interfaces**: an upstream microservice publishes data to an **event log (e.g., Kafka)** and downstream microservices subscribe, without the upstream service being tightly coupled to subscribers.

3.10 Chapter 3 Summary

- SEED(S) is an effective, repeatable methodology for designing robust microservices (and APIs).
- Order of work: **actors, then jobs, then interactions, then actions and queries, then an open-standard specification, then feedback, then implementation.**
- Chapters 4 and 5 go deeper into design (service boundaries and data); Chapter 9 implements sample microservices.

Quick Revision Sheet

Key terms

Term	Meaning (as used in the textbook)
Microservice	An independently deployable component of bounded scope that supports interoperability through message-based communication.
Coordination cost	The cost of teams aligning their work and dependencies; reducing it is the key benefit of microservices.
Analysis paralysis	Endless discussion of architectural choices out of fear of choosing wrong, caused by long feedback loops and complexity.
ADR / LADR	Architecture decision record; LADR is Michael Nygard's lightweight Markdown version (Status, Context, Decision, Consequences).
Operating model	The people, processes and tools underlying the system; the "operating system" of the solution.
Conway's Law	A system's design copies the organization's communication structure.
Inverse Conway maneuver	Designing teams and communication first so that the system architecture follows.
Stream-aligned team	Owns and runs a continual stream of deliverables ("You build it, you run it").
Enabling team	Consulting-style support from specialists.
Complicated-subsystem team	Owns a hard domain needing scarce expertise.
Platform team	Provides self-service capabilities treated as a product.
Collaboration / Facilitating / X-as-a-service	Interaction modes ranging from high to minimal coordination.
SEED(S)	Seven Essential Evolutions of Design for Services; the book's service design process.
Actor	The target consumer of a service, inspired by user personas; used to scope and prioritize.
JTBD / Job Story	Jobs to be done; captured as "When <circumstance>, I want to <motivation>, so I can <goal>".
CQS	Command query separation (Bertrand Meyer): actions change state; queries do not.
OAS	OpenAPI Specification, an open standard for describing RESTful APIs.
Microservice Design Canvas	Matt McLarty's alternative to actions-and-queries analysis for complex requirements.

Key decisions in Chapters 1 to 3

#	Decision
1	Use ADRs to log key decisions.

#	Decision
2	Limit team size (no more than eight people).
3	Define principles for team membership: cross-functional, only members who add value.
4	Design teams and coordination models before system architecture, and keep improving them.
5	Each microservice is owned by a single team for the service's lifetime.
6	Use a standard, repeatable process for service design.
7	Scope service design using key actors.
8	Use jobs as the unit of analysis.
9	Use the standard Job Story format.
10	Use PlantUML sequence diagrams to discover interaction patterns.
11	Separate service endpoints into commands and queries.
12	Collect feedback on service designs before considering design complete.
13	Layer web APIs on top of microservices; microservices are not just small APIs.